

ME 2261 - Embedded Systems Log BookTask 1

Toggle LED every 50 ms which is connected to PC0 by using Timer Zero (Timer 0). The CPU clock frequency is 8 MHz.

Methodology – Using interrupts

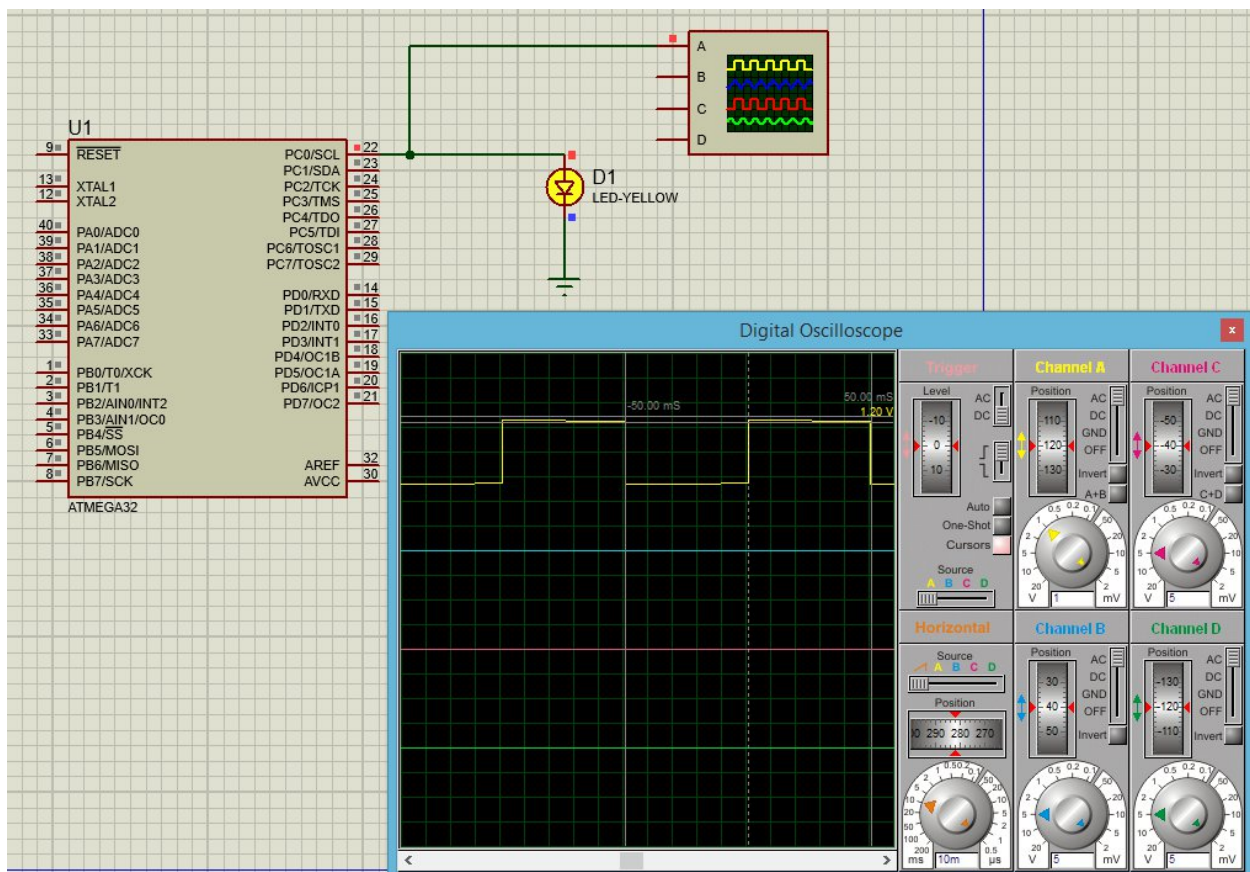
Prescalers	F clock	Timer count	
8	1 MHz	49999	Timer 0 is a 8 bit timer counter.
64	125 KHz	6249	
256	31.25 KHz	1561.5	Not an integer num
1024	7812.5 Hz	389.625	

Prescaler = 64, Timer Counter = 6249

Timer 0 is the 8 bit counter. After every 256 tick timer overflow. Therefore, timer has overflow 24 times.

Remaining time count = $6249 - (24 \times 256) = 105$

Thus, in the 25th iteration, we only allow the timer to count up to 105, and then reset it.



```

#define F_CPU 8000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

volatile uint16_t tot_overflow;

void timer0_init()
{
    TCCR0 |= (1 << CS01)|(1 << CS00);    // prescaler = 64
    TCNT0 = 0;    // Initialize counter
    TIMSK |= (1 << TOIE0);    //
    sei();    // enable global interrupt
    tot_overflow = 0;    // set number of overflow to 0
}

ISR(TIMER0_OVF_vect){
    tot_overflow++; // keep a track of number of overflows
}

int main(void){

    DDRC |= (1 << 0);    // connect led to pin PC0
    timer0_init();
    while(1){
        if (tot_overflow >= 24)    // check the number of overflows = 24
        {
            if (TCNT0 >= 105 )    // check if the timer count reaches 105
            {
                PORTC ^= (1<<0);    // toggles the led
                TCNT0 = 0;    //reset counter
                tot_overflow = 0;    // reset overflow count
            }
        }
    }
}

```

Task 2

Using TIMER0 generate the following frequency signal. Methodology using Interrupts.

On – 10 ms

For prescaler 64, timercounter = 1249, timer has overflow 4 times.

Remaining time count = $1249 - (4 * 256) = 225$

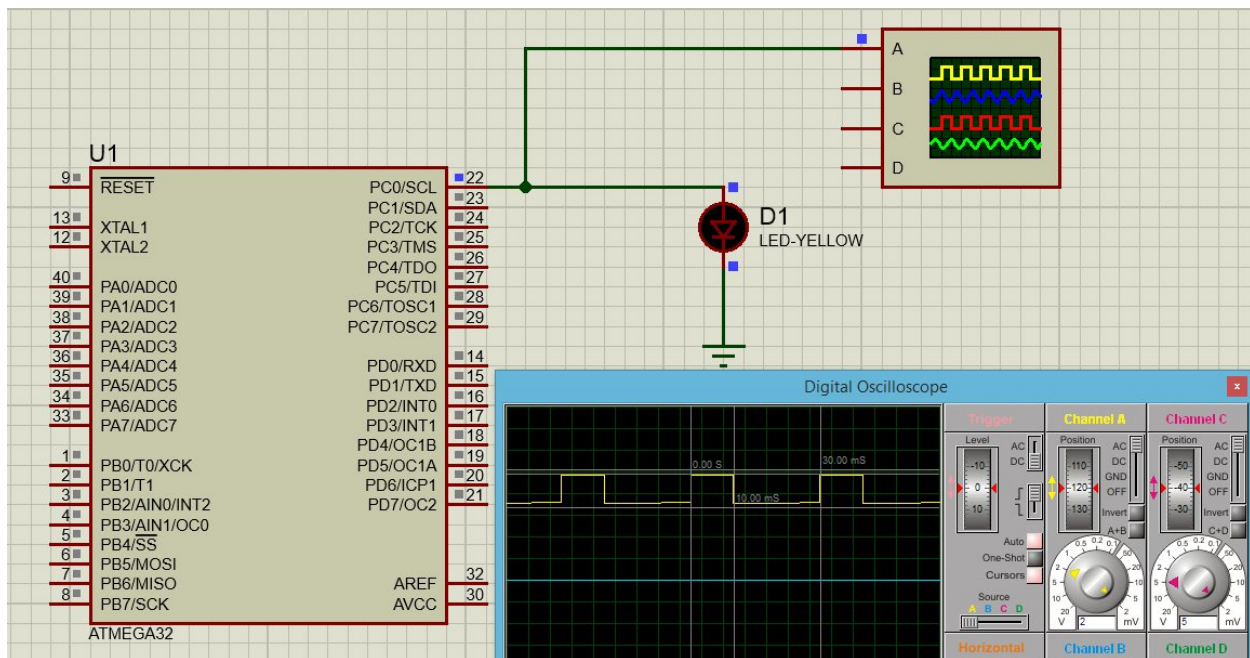
In 5th iteration, we have to count until 225, and reset it.

Off – 20 ms

For prescaler 64, timercounter = 2499, timer has overflow 4 times.

Remaining time count = $2499 - (9 * 256) = 195$

In 10th iteration, we have to count until 195, and reset it.



```

#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

volatile uint16_t tot_overflow;

void timer0_init()
{
    TCCR0 |= (1 << CS01)|(1 << CS00);    // prescaler = 64
    TCNT0 = 0;    // Initialize counter
    TIMSK |= (1 << TOIE0);    //
    sei();    // enable global interrupt
    tot_overflow = 0;    // set number of overflow to 0
}

ISR(TIMER0_OVF_vect){
    tot_overflow++; // keep a track of number of overflows
}

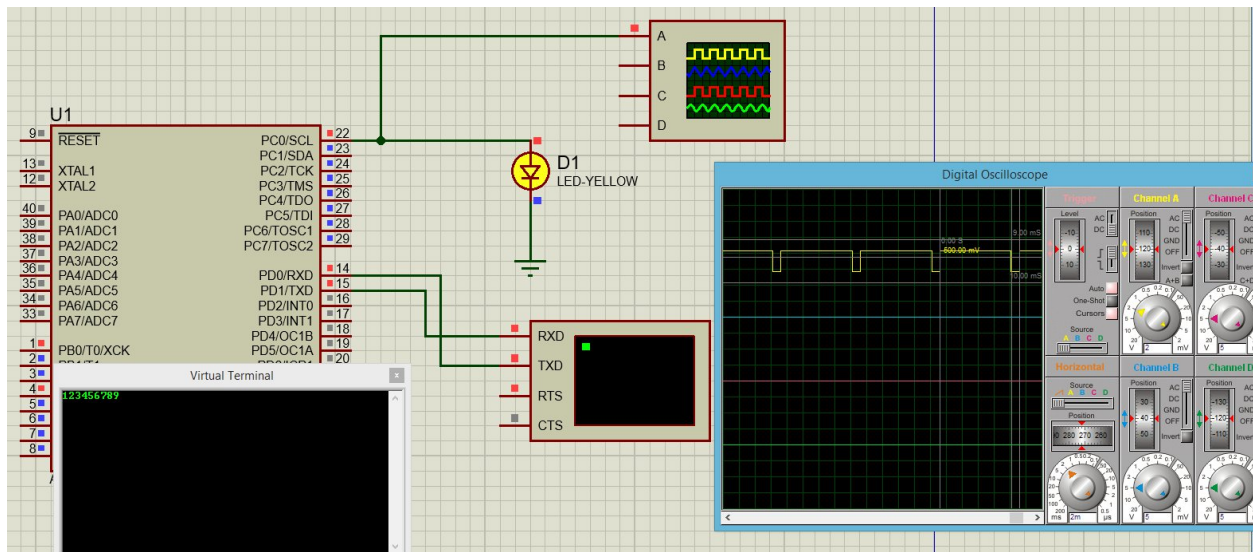
int main(void){
    DDRC |= (1 << 0);    // connect led to pin PC0
    timer0_init();
    while(1){
        if (tot_overflow >= 4 && PORTC == 1)
        {
            if (TCNT0 >= 225 )
            {
                PORTC = (0<<0); // On the led
                TCNT0 = 0;    //reset counter
                tot_overflow = 0;    // reset overflow count
            }
        }
        if (tot_overflow >= 9 && PORTC == 0)
        {
            if (TCNT0 >= 195 )
            {
                PORTC = (1<<0); // Off the led
                TCNT0 = 0;    //reset counter
                tot_overflow = 0;    // reset overflow count
            }
        }
    }
}

```

}

Task 3

Generate 100 Hz signal whose on-time(ms) is set by user input (0-9).
Methodology – Using interrupts and Timers



```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

volatile uint16_t tot_overflow;

void USART_init(void) {
    UCSRA = 0x00;
    UCSRB = 0x98;
    USRC = 0x86;
    UBRRL = 0x33;
    UBRRH = 0x00;
}
```

```

void USART_send(unsigned char data){
    while ((UCSRA & (1<<UDRE))==0x00);
    UDR = data;
}

unsigned char USART_receive(void){
    while ((UCSRA & (1<<RXC))==0x00);
    return UDR;
}

ISR(USART_RXC_vect) {
    unsigned char data = USART_receive();
    USART_send(data);
    PORTB = data-'0';      // Convert ASCII to Number
}

void timer0_init()
{
    TCCR0 |= (1 << CS01)|(1 << CS00);    // prescaler = 64
    TCNT0 = 0;      // Initialize counter
    TIMSK |= (1 << TOIE0);  //
    sei();    // enable global interrupt
    tot_overflow = 0;      // set number of overflow to 0
}

ISR(TIMER0_OVF_vect){
    tot_overflow++; // keep a track of number of overflows
}

void step_count(int num, int *iteration, int *steps) {
    int count = num * 125 - 1;  // for prescaler 64
    *iteration = count / 256;
    *steps = count % 256;
}

int main(void){
    USART_init();
    sei();
    DDRB = 0xFF;
    DDRC = 0xFF;
    DDRC |= (1 << 0);      // connect led to pin PC0
    timer0_init();
    while(1){
        int num = PORTB;
        int on_i, on_s, off_i, off_s;
        if (PORTB>0 && PORTB<10){
            step_count(num, &on_i, &on_s);
            step_count(10-num, &off_i, &off_s);
            if (tot_overflow >= on_i && PORTC == 1)
            {

```



```

}

void step_count(int num, int *iteration, int *steps) {
    int count = num * 125 - 1; // for prescaler 64
    *iteration = count / 256;
    *steps = count % 256;
}

ISR (INT0_vect)
{
    if (PORTB!=10){PORTB = PORTB+1;}
    else
        PORTB=0;
}

ISR (INT1_vect)
{
    if (PORTB!=0){PORTB = PORTB-1;}
    else
        PORTB=10;
}

int main(void){
    sei();
    DDRB = 0xFF;
    GICR = (1<<INT1) | (1<<INT0);
    MCUCR = (1<<ISC10) | (1<<ISC11);
    int num = 0;
    DDRC = 0xFF;
    DDRC |= (1 << 0); // connect led to pin PC0
    timer0_init();
    do{
        num = PORTB;
        int on_i, on_s, off_i, off_s;
        step_count(num, &on_i, &on_s);
        step_count(10-num, &off_i, &off_s);
        if (tot_overflow >= on_i && PORTC == 1)
        {
            if (TCNT0 >= on_s )
            {
                PORTC = (0<<0); // On the led
                TCNT0 = 0; //reset counter
                tot_overflow = 0; // reset overflow count
            }
        }
        if (tot_overflow >= off_i && PORTC == 0)
        {
            if (TCNT0 >= off_s )
            {
                PORTC = (1<<0); // Off the led
            }
        }
    } while(1);
}

```



```

        TCNT0 = 0;           //reset counter
        tot_overflow = 0;    // reset overflow count
    }
}
}while(PORTB<11);
}

```

